

React HandsOn 4: Life Cycle Hooks Fetch Posts

Objectives

Explain the need and Benefits of component life cycle

Identify various life cycle hook methods

List the sequence of steps in rendering a component

In this hands-on lab, you will learn how to:

Implement `componentDidMount()` hook

Implementing `componentDidCatch()` life cycle hook.

Prerequisites

The following is required to complete this hands-on lab:

Node.js

NPM

Visual Studio Code

Notes

Estimated time to complete this lab: 60 minutes.

Create a new react application using create-react-app tool with the name as "blogapp"

Open the application using VS Code

Create a new file named as `Post.js` in `src` folder with following properties

Figure 2: Post class

Create a new class based component named as `Posts` inside `Posts.js` file

Figure 3: Posts Component

Initialize the component with a list of `Post` in state of the component using the constructor

Create a new method in component with the name as `loadPosts()` which will be responsible for using `Fetch` API and assign it to the component state created earlier. To get the posts use the url (<https://jsonplaceholder.typicode.com/posts>)

Figure 4: `loadPosts()` method

Implement the `componentDidMount()` hook to make calls to `loadPosts()` which will fetch the posts

Figure 5: `componentDidMount()` hook

Implement the `render()` which will display the title and post of posts in html page using heading and paragraphs respectively.

Figure 6: `render()` method

Define a `componentDidCatch()` method which will be responsible for displaying any error happening in the component as alert messages.

Figure 7: `componentDidCatch()` hook

Add the `Posts` component to `App` component.

Build and Run the application using `npm start` command.